Cambridge O Level Computer Science

Chapter 7: Algorithm design and problem solving

Lecture notes with Check-Yourself questions and a full answer key.

In this chapter you will learn about:

- The program development life cycle: analysis, design, coding and testing.
- Computer systems and sub-systems; decomposing problems into component parts; designing solutions with structure diagrams, flowcharts and pseudocode.
- The purpose of an algorithm; standard methods of solution (linear search, bubble sort, totalling, counting, maximum, minimum, average).
- Validation and verification checks on input data; types of test data; trace tables and dry runs; writing, amending and correcting errors in flowcharts, pseudocode and programs.

An algorithm is an ordered set of steps to solve a problem. This chapter teaches you to design algorithms rigorously, test them with trace tables, and fix their errors, using the same standard methods you will program in Chapter 8.

7.1 The program development life cycle

The program development life cycle has five stages: analysis, design, coding, testing and maintenance. This chapter and Chapter 8 cover the first four.

7.1.1 Analysis

Before solving a problem, it must be clearly defined so everyone understands what is needed: the requirements specification. Analysis uses two tools:

- Abstraction: keeping the key elements needed for the solution and discarding unnecessary details. For example, a map shows only what is needed for a journey; different transport needs different maps.
- Decomposition: breaking a complex problem into smaller parts, then subdividing those into even smaller parts that are easy to solve. For example, getting dressed becomes: select items to wear; remove clothes being worn; put selected items on in order.

sakib sir

7.1.2 Design

The program specification from analysis is used to show how the program should be developed. When design is complete, the programmer knows what is to be done: all tasks, how each is performed, and how the tasks work together. This is formally documented with structure charts, flowcharts and pseudocode (see 7.2).

7.1.3 Coding and iterative testing

Each module of the program is written in a suitable programming language and tested. Iterative testing means: test a module, amend the code, repeat the tests until the module performs as required.

7.1.4 Testing

The completed program is run many times with different sets of test data, ensuring all tasks work together as specified in the design.

Check yourself

1. 1. Name the five stages of the program development life cycle, and state which four this chapter covers.
2. 2. What is a requirements specification?
3. 3. Explain the difference between abstraction and decomposition, giving an example of each.
4. 4. What should a programmer know once the design stage is complete?
5. 5. What is iterative testing?

7.2 Computer systems, sub-systems and decomposition

7.2.1 The computer system and its sub-systems

A computer system is made up of software, data, hardware, communications and people; each system can be divided into sub-systems, each sub-system into smaller sub-systems, until each one performs a single action. Systems range from tiny (an alarm app) to huge (a weather-forecast system).

Top-down design is this decomposition into sub-systems, then smaller sub-systems, until each performs a single action; the stepwise process is called stepwise refinement. Structure diagrams show it diagrammatically, hierarchically. It suits

sakib sir

large and small systems: several programmers can develop and test different sub-systems independently at the same time, reducing development and testing time. Each sub-routine's working can be shown with flowcharts or pseudocode.

7.2.2 Decomposing a problem

Every problem is decomposed into four component parts:

- Inputs: data entered while the system is active.
- Processes: tasks performed using the input data and any stored data.
- Outputs: information displayed or printed for users.
- Storage: data kept in files on a suitable medium for future use.

Example: an alarm app. Inputs: time to set the alarm, removing a set alarm, switching an alarm off, pressing snooze. Processes: continuously checking whether the current time matches a set alarm time, storing and removing alarm times, managing snooze. Outputs: continuous sound or tune at alarm time (or after snooze expires). Storage: the set alarm times.

Check yourself

6. 1. What five things make up a computer system?
7. 2. What is top-down design, and what is stepwise refinement?
8. 3. Give two benefits of top-down design when developing a large computer system.
9. 4. Decompose a microwave oven's cooking system into inputs, processes, outputs and storage.

7.2.3 Methods used to design and construct a solution

IGCSE students must use three formal methods: structure diagrams, flowcharts and pseudocode.

Structure diagrams

Structure diagrams show top-down design hierarchically: each level gives a more detailed breakdown of the system into sub-systems. Example: an alarm app divides into Set alarm, Check time and Sound alarm; Set alarm further divides into Set time and Turn alarm on/off; Check time into Check and Reset/clear alarm; Sound alarm into Play sound for two minutes and Turn off/snooze.

Check yourself

sakib sir

10. 1. What is a structure diagram used for?
11. 2. Draw a structure diagram for a washing machine system with at least three sub-systems, dividing one of them further.

Flowcharts

A flowchart shows diagrammatically the steps required to complete a task and their order: these steps plus their order are the algorithm. Flowcharts communicate how a system or sub-system works. Example: the alarm-time check: START, Get Time, decision 'Time = Alarm Time?' (No: Wait 30 seconds, loop back; Yes: Sound Alarm), STOP.

Standard flowchart symbols:

Symbol	Use	Example
Terminator (rounded rectangle)	Beginning and end of each flowchart	START / STOP
Process (rectangle)	Actions, for example assigning values to variables; a named process defined elsewhere uses a rectangle with double vertical edges	A <- 0
Input/output (parallelogram)	Input of data and output of information	INPUT X / OUTPUT "Error"
Decision (diamond)	Choosing the next action (selection or repetition); always has two outputs, both clearly labelled	x > B? (yes/no)
Flow lines (arrows)	Direction of flow, usually top to bottom and left to right	->

Example: concert ticket sales. Tickets cost $20 each; 10 tickets give 10% discount; 20 tickets give 20% discount; no more than 25 tickets per transaction. The flowchart: START; OUTPUT 'How many tickets would you like to buy?'; INPUT N; decision 'N >= 1 AND N < 26?' (No: back to input); decision 'N < 10?' (Yes: D <- 0; No: decision 'N < 20?' (Yes: D <- 0.1; No: D <- 0.2)); Cost <- N * (1 - D); OUTPUT 'Your tickets cost ', Cost; STOP.

Check yourself

salihosir